

Chapter 1

The Spatio-Temporal Traffic Forecaster

1.1 Introduction

The digital twin of Chapter ?? consumes an *offered load* and reports what a UPF would do at that load. It does not say where the load comes from. This chapter builds the component that does: a spatio-temporal forecaster that predicts near-future traffic for each of the K geographic service areas into which a city’s base stations are grouped.

As in the rest of this thesis, it is worth being precise about scope, because the system is deliberately decomposed into independent parts:

The forecaster is not a calibrator. It emits a *dimensionless* per-cluster load $\hat{d} \in [0, 1]$. The mapping to physical Gbps is the scenario constant α owned by the twin (Chapter ??, Eq. 1.1); it is applied *after* forecasting and never enters training. One trained forecaster therefore serves every capacity scenario.

The forecaster is not a controller. It predicts load; it does not choose a UPF configuration. Controllers consume the forecast (and its error bar) but are out of scope here (Chapter ??).

This separation mirrors the repository structure: the forecaster is a standalone project whose only coupling to the twin is a fixed file contract (Section 1.4).

The contribution of this chapter is not a new forecasting architecture but an evidence-based answer to two questions a system designer must settle

before wiring a forecaster to the twin. First, at *what spatial granularity* should one forecast: is it better to aggregate base stations into K service areas and forecast the cluster signal directly (*cluster-first*), or to forecast every base station and sum? Section 1.9 answers this with a controlled ablation and finds the cluster-first route dramatically better — aggregation *creates* predictability by denoising sparse per-station traffic. Second, *which forecaster* to use: Section 1.6 benchmarks six models from a linear baseline to established spatio-temporal graph networks under a common protocol, and finds the field tight, with a simple, cheap model losing little to the most sophisticated. The reusable substrate behind both is the cluster-first pipeline itself, and the headline finding is that on this problem the spatial structure that matters is captured chiefly by the *geographic aggregation*, with the inter-cluster graph adding a small further gain.

Place in the thesis. Within this thesis’s theme of *augmented observability* for the 5/6G data plane, the forecaster extends the observability envelope along its *temporal* axis. The profiling campaign made per-function power and QoS observable at the present instant; the forecaster pushes the horizon forward, making the *near future* of per-cluster load observable one control step ahead. A controller that could see only the present would have to react to congestion after it arrived; forecasting is what lets the downstream twin and controllers act before the load does. Its accuracy is therefore not a private metric but part of the observability contract: the twin sizes its anti-flapping band directly from the forecast error, so forecast quality propagates into control behaviour (Section 1.4).

The chapter proceeds as follows. Section 1.2 builds the forecasting graph from raw NetMob traffic. Section 1.3 introduces the forecasting models. Section 1.4 states the output contract consumed by the twin. Section 1.5 defines the evaluation methodology and the skill metric. Section 1.6 benchmarks the forecasting models and selects one; Sections 1.7–1.8 characterise the chosen model across cluster counts and traffic types; and Section 1.9 validates the cluster-first decision itself as the capstone. Section 1.10 summarises.

1.2 From Base Stations to a Forecasting Graph

Dataset. The forecaster is driven by NetMob 2023, the same source the twin’s NetMob adapter targets: per-tile downlink traffic for Lyon over 77 days (2019-03-16 to 2019-06-01) at 15-minute resolution (96 slots/day), for the services Netflix, YouTube, and DailyMotion. The raw values are privacy-

preserving aggregate indicators with *no physical unit*; this is the structural reason the pipeline is normalised and the twin needs α .

Normalisation. Tile traffic is aggregated to base stations and divided by a single global maximum computed over all services,

$$\hat{d} = \frac{\text{bs_tile_sum}}{\text{global_max}}, \quad \text{global_max} = 1.482 \times 10^8, \quad (1.1)$$

so $\hat{d} \in [0, 1]$. A *single* normaliser (rather than one per service) is used precisely so that the cross-service ratios are preserved and cluster signals can be formed by direct summation; empirically the mean normalised load of Netflix is $\approx 5.8\times$ that of DailyMotion, a difference the normalisation retains.

Tiles to base stations. Each 100 m tile is assigned to its containing cell of a Voronoi tessellation built from ~ 965 gNodeB locations (Cartoradio), giving a per-base-station downlink series. Two base stations are adjacent when their Voronoi cells share a border.

Clustering into K nodes. 965 base stations is too many for tractable per-area control, so base stations are grouped into K contiguous clusters with SKATER (a geographic minimum-spanning-tree partition), with a minimum cluster size of 3. SKATER is purely geographic: the cluster *structure* does not depend on the traffic service, only on base-station geometry — a property exploited in Section 1.8.

Coarsening. Each cluster becomes one graph node. Two clusters are connected when any member base station of one is Voronoi-adjacent to any member of the other; the edge weight is the mean inter-cluster affinity. The node signal at each 15-minute slot is the *sum* of member base-station $\hat{d}$. This produces a K -node graph and a $(K \times T)$ signal matrix that are the forecaster’s input.

1.3 Forecasting Models

Every model predicts, for all nodes simultaneously, the next $H = 4$ slots (1 h) from the previous $L = 96$ slots (24 h): input (B, L, N) of normalised loads (one feature per node) plus, for graph models, the static graph (edge_index, edge_attr);

[DRAW.IO PLACEHOLDER]

Forecasting pipeline / data flow. Left-to-right block diagram. (1) *Raw NetMob*: 100 m tile grid over Lyon, 15-min slots (icon: heatmap of tiles). (2) *Voronoi assignment*: tiles $\rightarrow$ ~ 965 gNodeBs (icon: Voronoi cells over a map, base-station dots). (3) *Normalise*: $\hat{d} = \text{bs_sum} / \text{global_max} \in [0, 1]$. (4) *SKATER clustering*: 965 base stations $\rightarrow K$ contiguous clusters (icon: map recoloured into K regions). (5) *Coarsen*: each cluster = one node; edges where clusters share a Voronoi border; node signal = sum of member $\hat{d}$. (6) *STGNN*: K -node graph + 24 h history $\rightarrow$ 1 h-ahead per-cluster $\hat{d}$. Show the output arrow leaving to “Digital Twin (Ch. ??)” to emphasise the hand-off. Mark clearly that everything left of the STGNN is one-time preprocessing and the cluster structure is shared across services.

Figure 1.1: The cluster-first forecasting pipeline: raw tiles are aggregated to base stations, clustered geographically into K nodes, and forecast at the cluster level.

output (B, H, N) , with $N = K$ clusters at the cluster level. To answer “which forecaster?” without favouring any paradigm, the benchmark of Section 1.6 spans, under one training protocol:

Naive floors. *Persistence* (repeat the last value) and *seasonal-naive* (the same slot 24 h earlier) — the references the skill metric is measured against.

DLinear [4] — a channel-independent trend/seasonal decomposition with two linear maps; the “is anything deep needed?” baseline.

GRU (no graph) — a per-node gated recurrent encoder; the purely temporal model, and the natural ablation of the graph (any graph model’s edge over it is the value of spatial information).

DCRNN [1] — diffusion-convolutional recurrence; spatial mixing fused into a GRU cell over the given graph.

Graph WaveNet [2] — dilated temporal convolutions with a learnable adjacency.

AGCRN [3] — a node-adaptive GRU over an adjacency the model *learns* from data, ignoring the supplied graph (a test of whether a better graph than Voronoi exists).

[DRAW.IO PLACEHOLDER]

Example spatio-temporal architecture (the in-house GAT-GRU).

Input tensor $(B, L=96, N)$ on the left. Lane 1 “Temporal”: per-node GRU (hidden 64) over the 96-step series $\rightarrow$ node embeddings $(B, N, 64)$. Lane 2 “Spatial”: 2-layer GAT (4 heads) over the N -node graph, drawn as nodes with weighted edges and little attention-head fans. “Output head”: linear $\rightarrow (B, H=4, N)$. Annotate the static graph inputs (edge_index, edge_attr) feeding the GAT. Keep colours consistent with Figure 1.1. This figure is illustrative of the STGNN family; the other graph models differ in how spatial and temporal mixing are composed.

Figure 1.2: An example spatio-temporal forecaster (the in-house GAT-GRU): a per-node GRU temporal encoder feeding a graph-attention spatial mixer and a linear forecast head.

GAT-GRU (in-house) — a per-node GRU temporal encoder followed by a two-layer graph-attention spatial mixer (4 heads) and a linear head; included as a reasonable in-house spatio-temporal baseline, not as a proposed method.

The graph models share the same coarse K -node graph; the GRU and DLinear ignore it. All are *compact reimplementations on a common harness*: results reflect each architecture’s inductive bias under identical data, budget, and protocol rather than its best-tuned reference performance — the appropriate footing for a controlled comparison.

1.4 Outputs and the Twin Data Contract

The forecaster does not call the twin; it writes files the twin reads. For a chosen (service, K) it exports seven artifacts into a flat directory consumed by the twin’s loader: test-window predictions and targets (`predictions_test.npy`, `targets_test.npy`), the full cluster signal (`cluster_series.npy`), the cluster $\rightarrow$ base-station map and assignment table, the base-station coordinates, and the evaluation summary (`forecast_eval_summary.json`). The arrays are in normalised $\hat{d} \in [0, 1]$; the twin applies α . Critically, the forecaster’s reported mean absolute error is part of the contract: the twin derives its anti-flapping hysteresis band as $2 \cdot \text{MAE}_{\text{forecast}}$ (Chapter ??, Eq. 1.6), so forecast accuracy propagates directly into control behaviour.

1.5 Evaluation Methodology

Protocol. The data is split chronologically (70/15/15 train/val/test; no shuffling) to prevent leakage. Models train with Adam, an L1 (MAE) objective, cosine learning-rate decay, and early stopping on validation MAE (patience 15). Unless stated, results are averaged over three seeds.

Metrics. Forecasts are scored in $\hat{d}$ units with MAE, RMSE, and WAPE,

$$\text{WAPE} = 100 \cdot \frac{\sum |y - \hat{y}|}{\sum |y|}, \quad (1.2)$$

which is used instead of MAPE because traffic is sparse and MAPE explodes on near-zero denominators.

Skill, not raw error. Absolute error is misleading across graph sizes: as K grows, each cluster aggregates fewer base stations, so its signal is burstier and *intrinsically* harder in relative terms even though its absolute magnitude shrinks. The model-independent quantity is therefore *skill* over a baseline,

$$\text{skill} = 1 - \frac{\text{WAPE}_{\text{model}}}{\text{WAPE}_{\text{baseline}}}, \quad (1.3)$$

reported against persistence (last value) and seasonal-naive (same slot 24h earlier); a historical-mean baseline is also computed. Skill isolates what the model contributes from how hard the target intrinsically is.

1.6 Which Forecaster? A Model Benchmark

The system places one forecaster per MEC cluster (Section 1.2), so the first operative question is: which model best forecasts at the cluster level? Table 1.1 and Figure 1.3 benchmark six learned forecasters (Section 1.3) under one protocol at $K \in \{10, 50\}$ on Netflix.

Three findings stand out.

First, spatial information helps — modestly. Every model that uses the given graph or learns one (Graph WaveNet, DCRNN, AGCRN) beats the graph-free GRU at both sizes (0.253/0.258/0.243 vs. 0.234 at $K=10$): a real, repeatable $\sim 3\%$ WAPE reduction. But the gain is small and does not grow with K , and *learning* the adjacency (AGCRN) does not beat the given Voronoi graph — so the ceiling is the weak residual spatial signal at cluster level, not the graph’s quality. Section 1.9 explains why: the geographic

Table 1.1: Model comparison at $K=10$ and $K=50$ (Netflix, seed-averaged over 3 seeds). Skill is over persistence (higher is better); WAPE in %. Best skill per column in bold; training time ranges over the two graph sizes and seeds. All models are compact reimplementations under a common protocol.

model	params	$K=10$		$K=50$		train (s)
		WAPE	skill	WAPE	skill	
persistence	—	29.5	0.000	44.4	0.000	—
DLinear	0.8k	23.6	0.199	36.2	0.185	4–7
GRU (no graph)	13k	22.6	0.234	34.8	0.217	15–60
Graph WaveNet	78k	22.0	0.253	34.2	0.229	50–170
DCRNN	75k	21.9	0.258	33.9	0.236	650–1040
AGCRN	252k	22.3	0.243	34.1	0.233	1670–1790
GAT-GRU (in-house)	60k	23.0	0.219	35.0	0.211	120–680

aggregation has already captured most of the spatial structure, leaving little for the inter-cluster graph to add.

Second, the field is tight and simple models lose little. From DLinear (0.199) to the best model (0.258) the entire spread is ~ 0.06 skill, and the no-graph GRU (0.234) is within ~ 0.02 of the best. Model choice barely moves the needle.

Third, accuracy-cost favours the simple. DCRNN leads but is among the slowest; Graph WaveNet matches it to within 0.005 skill at roughly a tenth of the cost (its dilated convolutions parallelise where DCRNN’s recurrence does not); AGCRN is the costliest (252k parameters, ~ 28 min/run) for no gain; and the in-house GAT-GRU does not lead. We therefore adopt **Graph WaveNet** as the deployed forecaster (best accuracy-cost); a plain **GRU** would be a defensible simpler choice. The rest of the chapter characterises Graph WaveNet, and Section 1.9 shows the one decision the system is *not* insensitive to — forecasting granularity — dwarfs every model difference here.

1.7 Effect of Cluster Count K

The number of MEC clusters K is a deployment parameter — how finely the operator partitions the city. How does the chosen forecaster (Graph WaveNet) behave as K grows from a handful of clusters to one hundred? Table 1.2 sweeps $K \in \{4, \dots, 100\}$ (Netflix, 3 seeds).

Across a $25\times$ change in K , Graph WaveNet holds a near-constant ~ 0.23 –

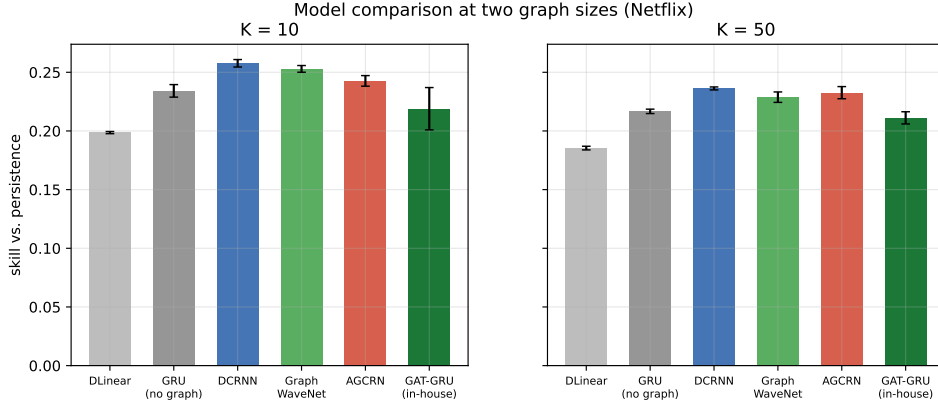


Figure 1.3: Skill over persistence of the six learned forecasters at $K=10$ (left) and $K=50$ (right); error bars are ± 1 seed std. The whole field spans only ~ 0.06 skill: the graph models lead the graph-free GRU by a small margin, and even a linear model is close behind.

0.27 skill over persistence and ~ 0.33 over seasonal-naive, even as raw WAPE climbs from 16.4 to 38.8 and MAE falls from 0.192 to 0.018. The rising WAPE and falling MAE are two views of one intrinsic effect — smaller clusters aggregate fewer stations, so their signal is burstier in *relative* terms but smaller in *absolute* terms — while the flat skill curve shows the model keeps the same advantage regardless of granularity (this is precisely why skill, not raw WAPE, is the headline metric). For the operator the practical consequence is that K can be chosen on deployment grounds — number of MEC sites, controller tractability — without paying a forecasting penalty.

1.8 Generalization Across Traffic Types

Because SKATER clustering is geographic, the *same* cluster structure can be re-used for a different service, changing only the signal on the nodes. This is a clean external-validity test: does Graph WaveNet’s performance hold for traffic with a different spatial and temporal profile? Table 1.3 reports skill over persistence for the aggregate of all services (**total**) and the sparse **DailyMotion**, alongside Netflix, at $K \in \{8, 10, 15\}$ (3 seeds, frozen architecture).

Skill is positive for every service and every K , so the result is not Netflix-specific: it holds for aggregate and sparse traffic on the same geographic

Table 1.2: Graph WaveNet forecast quality vs. cluster count K (Netflix, seed-averaged). Raw WAPE rises and MAE falls with K — both intrinsic to shrinking clusters — while *skill* over the baselines stays flat, the signature of a model that scales.

K	test WAPE	test MAE	skill vs. persist.	skill vs. seasonal
4	16.4	0.192	0.266	0.333
6	18.9	0.147	0.250	0.328
8	20.5	0.120	0.248	0.329
10	22.0	0.103	0.253	0.337
15	24.8	0.078	0.239	0.333
20	26.7	0.062	0.238	0.334
30	30.4	0.048	0.231	0.335
50	34.2	0.032	0.229	0.338
100	38.8	0.018	0.232	0.347

Table 1.3: Graph WaveNet skill over persistence by traffic type (seed-averaged). Positive for every service; the margin tracks predictability — largest on smooth aggregate traffic, smallest on sparse DailyMotion.

service	$K=8$	$K=10$	$K=15$
total (aggregate)	0.305	0.287	0.281
Netflix	0.248	0.253	0.239
DailyMotion (sparse)	0.140	0.160	0.153

graph. The margin behaves as predictability would suggest — widest on the smooth `total` signal (~ 0.29), narrower on Netflix, and smallest on the heavily sparse DailyMotion (~ 0.15 vs. persistence). DailyMotion is the instructive case: its persistence-skill is low because its traffic is bursty, yet its *seasonal*-skill is the highest of all (~ 0.37) — the sparse signal is very repetitive day-to-day, so the model beats persistence only modestly but beats seasonal-naïve strongly. The honest scope is robustness *across traffic types within Lyon*: the services share city, base stations, and period, so this is not evidence of cross-city transfer.

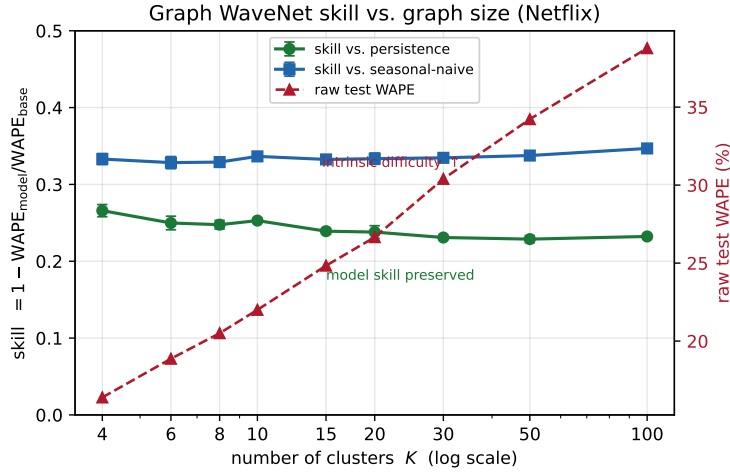


Figure 1.4: Graph WaveNet preserves its skill over the naive baselines from 4 to 100 clusters even as the raw error grows with the intrinsically harder small-cluster signal. The dashed red curve (raw WAPE, right axis) rises while the solid skill curves (left axis) stay flat.

1.9 Cluster-First vs. Per-Base-Station: Why Aggregate?

The preceding sections chose a model (Graph WaveNet) and showed it forecasts cluster load robustly across graph sizes and traffic types. All of it rests on one prior decision: that we forecast at the *cluster* level at all. The MEC deployment already dictates clusters — but is aggregating first also a *forecasting* advantage, or merely a convenience we would pay for in accuracy? This closing section tests that decision head-on.

Section 1.7 cannot answer it — it scores each K against its own (changing) target. A controlled test instead fixes the delivered target (the $K=10$ cluster signal) and compares two routes to it:

Route A (cluster-first). Aggregate to $K=10$ clusters; forecast the cluster signals directly.

Route B (per-base-station). Forecast all 965 base stations; *sum* the predictions into the same 10 clusters.

Both are scored on the identical $K=10$ target — the 965-station series sums exactly to the cluster series (verified to floating-point error) — so the com-

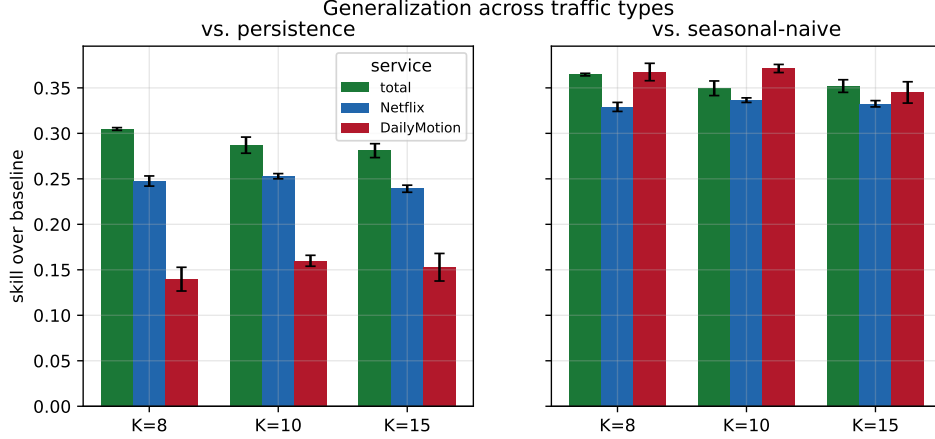


Figure 1.5: Graph WaveNet’s advantage over both baselines holds for aggregate and sparse traffic, scaling with the predictability of each service. The seasonal-naive panel (right) shows DailyMotion’s skill rising to ~ 0.37 , above Netflix, despite its sparsity.

parison isolates *when* aggregation happens. The route is run with the graph-free GRU and a sparse graph-attention STGNN; the dense-adjacency models (Graph WaveNet, DCRNN, AGCRN) do not even scale to 965 nodes, which is itself telling.

The result (Figure 1.6) is decisive: forecasting at the base-station level and summing yields a cluster forecast *no better than persistence* (skill -0.02 to -0.03), whereas forecasting the cluster signal directly reaches 0.21 – 0.23 — consistent across both models and all three seeds. The mechanism is *denoising by aggregation*: per-station traffic is extremely sparse (for Netflix, 85–99% of station-slots are below 0.01), so 965 such series are nearly unpredictable individually, and a sum of persistence-level forecasts is itself persistence-level. The *sum*, by contrast, is a smooth, strongly diurnal signal that is easy to forecast.

This reframes where spatial structure helps, and closes the chapter’s argument. Aggregation does not discard spatial information — it is the operation that *exploits* it, grouping geographically coherent stations into a predictable signal. That is why the inter-cluster graph adds only $\sim 3\%$ in Section 1.6 (the aggregation already did the heavy lifting), and why no model choice comes close to the gap seen here. **Cluster-first is a forecasting advantage, not merely a tractability convenience** — the central,

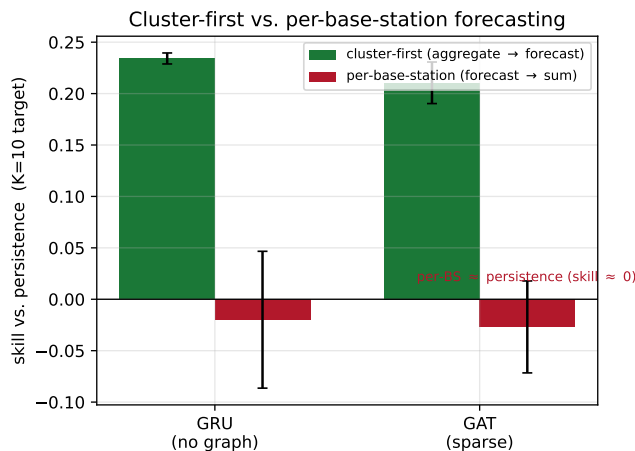


Figure 1.6: Forecasting the aggregated cluster signal directly (cluster-first) versus forecasting 965 base stations and summing (per-base-station), scored on the same $K=10$ target. The per-station route collapses to persistence-level skill (≈ 0); cluster-first reaches 0.21–0.23.

validated design decision of this chapter. (A far larger per-station model might recover some skill, but the gap — 0.22 versus ≈ 0 — is too large to plausibly close, and is consistent with the per-station sparsity.)

1.10 Summary

This chapter built the spatio-temporal forecaster that supplies the digital twin’s offered load. The principal points are:

1. **Scope.** The forecaster emits dimensionless per-cluster load $\hat{d} \in [0, 1]$; physical calibration (α) and control are external. One trained model serves every capacity scenario.
2. **Cluster-first is a forecasting advantage (central result).** A controlled ablation on a fixed target shows that forecasting the aggregated cluster signal directly reaches 0.21–0.23 skill, while forecasting all 965 base stations and summing is no better than persistence (≈ 0). The geographic aggregation *denoises* sparse per-station traffic into a predictable signal; it is the operation that exploits spatial structure, not one that discards it.

3. **Spatial structure helps modestly beyond aggregation.** On top of the clusters, the inter-cluster graph adds only a small gain ($\sim 3\%$ WAPE; graph models $>$ no-graph GRU), and a model that *learns* its adjacency (AGCRN) does not beat the given Voronoi graph — the residual spatial signal at cluster level is genuinely small.
4. **Model choice barely matters; deploy simple.** Six forecasters from linear to learned-graph span only ~ 0.06 skill; Graph WaveNet gives the best accuracy–cost trade-off and a plain GRU is near-best, so the system is insensitive to forecaster sophistication. Graph WaveNet is recommended for the twin.
5. **The problem is well-behaved.** Skill over persistence is near-constant across graph sizes ($K = 4 \rightarrow 100$; rising raw WAPE reflects intrinsic small-cluster difficulty, not model decay) and holds across aggregate and sparse traffic types — evidence the findings reflect Lyon’s spatial structure, not the Netflix trace.

Bibliography

- [1] Y. Li, R. Yu, C. Shahabi, Y. Liu. Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting. *ICLR*, 2018.
- [2] Z. Wu, S. Pan, G. Long, J. Jiang, C. Zhang. Graph WaveNet for Deep Spatial-Temporal Graph Modeling. *IJCAI*, 2019.
- [3] L. Bai, L. Yao, C. Li, X. Wang, C. Wang. Adaptive Graph Convolutional Recurrent Network for Traffic Forecasting. *NeurIPS*, 2020.
- [4] A. Zeng, M. Chen, L. Zhang, Q. Xu. Are Transformers Effective for Time Series Forecasting? *AAAI*, 2023.